TECHRISE 3.0

Cybersecurity & DevSecOps Programme · Abia Cohort 3.0 · 2026

PHASE 1 · FOUNDATION & RED TEAM

WEEK 03

Linux Essentials, Scripting, and Log Analysis

Student Training Manual — AI-First Curriculum

PROGRAMME	COHORT	WEEK	PHASE	METHODOLOGY
Cybersecurity & DevSecOps	**Abia Cohort 3.0**	**3 of 12**	**1 — Foundation & Red Team**	**AI-First (Manual → AI)**

Prepared by the TechRise Curriculum Design Team

Lead: Engr. Eduwem Adam · Co-facilitator: Mr. Akorita-Ifeanyichukwu Nehemiah

TABLE OF CONTENTS

- **Week 3: Linux Essentials, Scripting, and Log Analysis**
  - Phase: Foundation (Weeks 1–4)
  - Day 1 — Advanced Linux Commands for Security (grep, awk, sed, cut)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 2 — Bash Scripting for Security Automation
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 3 — Log Analysis from CLI (auth.log, syslog, Apache)
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 4 — Python Basics for Security + AI-Assisted Development
    - LEARN · PRACTICE · BUILD · DOCUMENT
  - Day 5 — Review, Assessment & Portfolio (Mini Project 1 Submission)
    - LEARN · PRACTICE · BUILD · DOCUMENT
- **Weekly Checklist (Student-Facing)**
- **Skills Verification**
- **Self-Assessment Quiz**
- **Additional Resources**
- **Looking Ahead — Week 4**

Phase: Foundation (Weeks 1–4) **Week 3 of 12**

WEEK 3: LINUX ESSENTIALS, SCRIPTING, AND LOG ANALYSIS

Weekly Overview

This week you stop being a passive consumer of security knowledge and become an active analyst. You will learn to extract intelligence from raw log files, automate repetitive security tasks with bash scripts, and use Python to scale your analysis. The manual skills you learn first — grep, awk, sed, log parsing — will then be enhanced with AI to find patterns you might miss.

How this week builds on Weeks 1–2

- Week 1 gave you Linux navigation and permissions. Week 2 gave you threat models and risk assessment. This week you apply both: you will write scripts that audit permissions (Week 1 skill) and analyse logs for attack patterns (Week 2 threat knowledge).
- Your CVE research skills from Week 2 become automated Python lookups.

How this week sets up Week 4

- Week 4 is reconnaissance, scanning, and exploitation. You cannot analyse scan results or understand exploit logs without log analysis skills.
- The scripts you write this week become the foundation for your Week 4 attack automation.

Why this matters in a real security job

- Security analysts spend 40–60% of their time reading logs and writing small automation scripts.
- The difference between a junior and senior analyst is not knowing more commands — it's knowing how to chain commands together to find answers faster. This week teaches that skill.

DAY 1

Advanced Linux Commands for Security (grep, awk, sed, cut)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Use grep to search log files for attack patterns

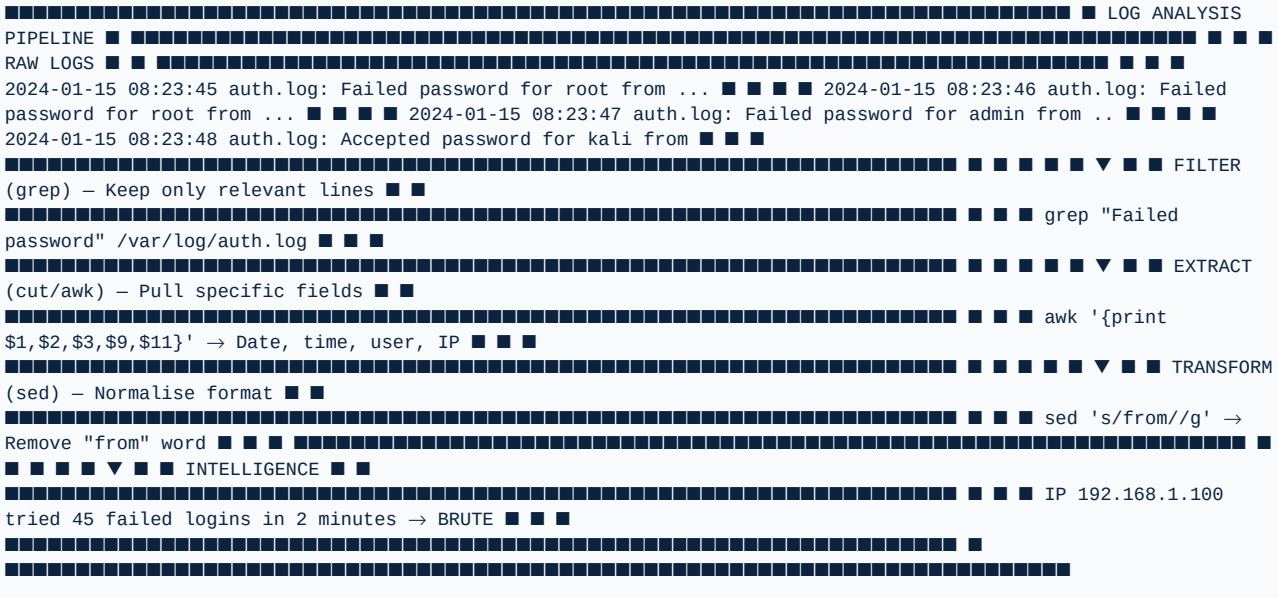
2. Extract specific columns from security data using cut and awk

3. Transform log output with sed for easier analysis

LOG ANALYSIS

is the process of examining system, application, and security logs to identify security incidents, troubleshoot issues, and detect anomalous behaviour.

Log Analysis Pipeline — Raw Log to Intelligence



Essential Log Analysis Commands

Command	Purpose	Security Use Case
`grep`	Search for patterns	Find all failed SSH attempts
`grep -v`	Exclude patterns	Remove noise (e.g., internal IPs)
`grep -c`	Count matches	How many attacks from an IP?

`cut`	Extract columns	Get just IP addresses from log lines
`awk`	Powerful text processing	Extract, reformat, sum, average
`sed`	Stream editor	Replace, delete, transform text
`sort`	Order lines	Group same IP addresses together
`uniq -c`	Count unique lines	Count occurrences per IP
`wc -l`	Count lines	Total failed attempts

grep Pattern Matching Examples

```
■■■
```

LAB 3.1.1 — Log Analysis with grep, awk, sed

Objective: Extract meaningful security intelligence from raw auth.log data.

Prerequisites: Terminal open; Kali Linux running

Step 1 — Create a Sample Log File for Practice

Copy this command exactly — it creates a realistic auth.log sample:

BASH

```
cat > ~/practice_auth.log << 'EOF' Jan 15 08:23:41 server sshd[1234]: Failed password for root from
192.168.1.100 port 52341 ssh2 Jan 15 08:23:42 server sshd[1234]: Failed password for root from
192.168.1.100 port 52342 ssh2 Jan 15 08:23:43 server sshd[1234]: Failed password for admin from
192.168.1.100 port 52343 ssh2 Jan 15 08:23:44 server sshd[1234]: Failed password for root from
10.0.0.5 port 22100 ssh2 Jan 15 08:23:45 server sshd[1234]: Failed password for root from
192.168.1.100 port 52344 ssh2 Jan 15 08:24:01 server sshd[1235]: Accepted password for kali from
192.168.1.50 port 55000 ssh2 Jan 15 08:24:15 server sshd[1234]: Failed password for root from
192.168.1.100 port 52345 ssh2 Jan 15 08:25:00 server sudo[2001]: kali : TTY=pts/0 ; PWD=/home/kali ;
USER=root ; COMMAND=/bin/bash Jan 15 08:30:00 server sshd[1234]: Failed password for oracle from
185.220.101.5 port 44123 ssh2 Jan 15 08:30:01 server sshd[1234]: Failed password for postgres from
185.220.101.5 port 44124 ssh2 EOF
```

Step 2 — Find All Failed Login Attempts

BASH

```
grep 'Failed password' ~/practice_auth.log
```

Expected: 8 lines. What you see is every failed login attempt.

Count them:

BASH

```
grep -c 'Failed password' ~/practice_auth.log
```

Expected output: 8

Step 3 — Find Top Attacking IP Addresses

BASH

```
grep 'Failed password' ~/practice_auth.log | awk '{print $11}' | sort | uniq -c | sort -rn
```

Expected output:

```
5 192.168.1.100 2 185.220.101.5 1 10.0.0.5
```

What this tells you: 192.168.1.100 tried 5 times — likely brute force. 185.220.101.5 tried multiple usernames — credential stuffing pattern.

Step 4 — Find Successful Logins — The Most Dangerous Log Entry

BASH

```
grep 'Accepted password' ~/practice_auth.log
```

Expected: One successful login from 192.168.1.50. Now check: was 192.168.1.50 also in the failed list? If yes — brute force succeeded.

Step 5 — Detect Privilege Escalation

BASH

```
grep 'sudo' ~/practice_auth.log | awk '{print $1,$2,$3,$6,$NF}'
```

What this tells you: User kali ran /bin/bash as root at 08:25. Combined with the failed logins before it, this is a complete attack chain in your log.

■ ANALYSE THIS

The log shows: 5 failed logins from 192.168.1.100, then 1 successful login from 192.168.1.50, then a sudo to root shell. Build a timeline of these events. Which OWASP category does this chain represent? What STRIDE threat type is the brute force?

BUILD *(2 Hours)*

BUILD BRIEF

Create a Log Analysis Command Reference Guide — your personal cheat sheet for analysing the three most common log files in security work.

Log 1 — /var/log/auth.log (SSH, sudo, login events):

- 5 grep commands with exact syntax and what each finds
- The complete brute force detection one-liner
- How to find successful logins after failed attempts (attack chain)

Log 2 — /var/log/syslog (system events):

- How to find service crashes (potential exploitation)
- How to find cron job execution (potential persistence)
- How to find unexpected process launches

Log 3 — Apache access.log (web traffic):

- How to find SQL injection attempts in GET requests
- How to find directory traversal (../../etc/passwd)
- How to find vulnerability scanner fingerprints

Format:

- Command → Purpose → Example Output → What it means → Red flag pattern

Quality standard:

- Every command tested and working on your Kali VM. Copy the actual output.

DOCUMENT

Writing Prompt:

Write a Security Incident Report for the events in the practice_auth.log file. Audience: IT Manager. Format: professional incident report.

- Required sections: Incident Summary (1 paragraph), Timeline of Events (bullet points with timestamps), Attack Methodology (which OWASP/STRIDE categories), Evidence (log extracts), Immediate Actions Taken, Recommended Remediation (3 actions)
- Classify the incident severity using the Likelihood × Impact matrix from Week 2

DAY 2

Bash Scripting for Security Automation

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Write a bash script with variables, conditionals, and loops

2. Automate log analysis tasks using bash pipelines

3. Build a reusable security audit script framework

BASH SCRIPTING

is writing a sequence of shell commands in a text file that can be executed automatically, allowing security analysts to automate repetitive tasks and scale their analysis across many systems.

Bash Script Anatomy — Every Security Script Needs These

BASH

```
#!/bin/bash # ============================================================ # Script Name:
security_audit.sh # Author: [Your Name] # Date: [Today] # Purpose: Automated security audit of local
system # Usage: sudo ./security_audit.sh #
============================================================ # ■■ Variables
■■■
```

Redirection	> file (overwrite) >> file (append)	echo $result >> $REPORT_FILE

Security-Specific Script Patterns

BASH

```
# ■■ Pattern 1: Count and Alert ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ FAILED=$(grep -c 'Failed password' $LOG_FILE 2>/dev/null || echo 0) if [ "$FAILED" -gt "$THRESHOLD" ]; then echo "[ALERT] $FAILED failed logins detected!" fi # ■■ Pattern 2: Extract and Report ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ echo "--- TOP ATTACKING IPs ---" grep 'Failed password' $LOG_FILE | \ awk '{print $11}' | \ sort | uniq -c | sort -rn | head -5 # ■■ Pattern 3: Check for Suspicious Files ■■■■■■■■■■■■■■■■■■■■■■ echo "--- SUID FILES (potential privesc) ---" find / -perm /4000 -type f 2>/dev/null | head -20 # ■■ Pattern 4: Active Network Connections ■■■■■■■■■■■■■■■■■■■■■■ echo "--- ESTABLISHED CONNECTIONS ---" ss -tuln | grep LISTEN # ■■ Pattern 5: Save and Timestamp ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ echo "Audit complete: $(date)" >> $REPORT_FILE echo "Report saved to: $REPORT_FILE"
```

WHY THIS MATTERS

The 2017 WannaCry ransomware spread to 200,000 machines in 150 countries within hours. Organisations with automated patch-checking scripts could have identified vulnerable systems (Windows SMBv1) before the attack. A 20-line bash script scanning for the affected service would have given defenders a 2-month head start after the vulnerability was published.

PRACTICE *(2 Hours)*

LAB 3.2.1 — Build Your First Security Script

Objective: Write a bash script that performs real security checks and generates a report.

Prerequisites: Terminal open; Kali Linux; text editor (nano or vim)

Step 1 — Create the Script File

BASH

```
nano ~/security_check.sh
```

Add the shebang and basic structure first — before writing any logic:

BASH

```
#!/bin/bash # TechRise 3.0 — Week 3 Lab # Security Check Script # Author: [Your Name] # Date: $(date) REPORT="/tmp/security_report_$(date +%Y%m%d).txt" echo "TechRise Security Audit" > $REPORT echo "========================" >> $REPORT echo "Date: $(date)" >> $REPORT echo "Host: $(hostname)" >> $REPORT echo "User: $(whoami)" >> $REPORT echo "" >> $REPORT
```

BASH

```
chmod +x ~/security_check.sh
```

Step 2 — Add System Information Section

BASH

```
echo "=== SYSTEM INFO ===" >> $REPORT uname -a >> $REPORT echo "" >> $REPORT echo "=== LOGGED-IN USERS ===" >> $REPORT who >> $REPORT echo "" >> $REPORT echo "=== LAST 5 LOGINS ===" >> $REPORT last | head -5 >> $REPORT echo "" >> $REPORT
```

Step 3 — Add Network Security Checks

BASH

```
echo "=== OPEN PORTS ===" >> $REPORT ss -tuln >> $REPORT echo "" >> $REPORT echo "=== ESTABLISHED CONNECTIONS ===" >> $REPORT ss -tun state established >> $REPORT echo "" >> $REPORT
```

Step 4 — Add Log Analysis Section with Alerting

BASH

```
echo "=== AUTH LOG ANALYSIS ===" >> $REPORT # Count failed logins FAILED=$(grep -c 'Failed password' /var/log/auth.log 2>/dev/null || echo 0) echo "Failed login attempts: $FAILED" >> $REPORT # Alert if threshold exceeded if [ "$FAILED" -gt 5 ]; then echo "[ALERT] HIGH number of failed logins!" >> $REPORT fi # Top attacking IPs echo "Top attacking IPs:" >> $REPORT grep 'Failed password' /var/log/auth.log 2>/dev/null | \ awk '{print $11}' | sort | uniq -c | sort -rn | head -5 >> $REPORT echo "" >> $REPORT
```

Step 5 — Run and Review

BASH

```
sudo ./security_check.sh cat /tmp/security_report_$(date +%Y%m%d).txt
```

■ ANALYSE THIS

Your script generates a report every time it runs. How would you schedule it to run automatically every hour? What Linux tool handles scheduled tasks? What security risk does an automated script running as root introduce?

BUILD *(2 Hours)*

BUILD BRIEF

Extend security_check.sh into a complete Security Audit Framework — a professional-grade script you could run on any Linux server.

Add these sections to your script:

- User Audit — list all users with UID 0 (root-equivalent), list users with empty passwords, list recently created users
- File System Audit — find SUID/SGID files, find world-writable directories, find files modified in the last 24 hours
- Process Audit — list all running processes, flag any process running from /tmp (red flag!)
- Cron Job Audit — list all cron jobs for all users
- Summary Section — count of alerts generated, overall risk assessment (LOW/MEDIUM/HIGH)

Script requirements:

- Must complete without errors (2>/dev/null for permission-denied noise)

- Must generate timestamped report file
- Must use at least one if/else conditional per section
- Must use at least one for or while loop

Deliverable:

- Working script + sample report output + 1-page explanation of what each section checks and why

DOCUMENT

Writing Prompt:

Write a Bash Scripting Best Practices Guide for your team. Audience: Security analysts who know bash basics but write messy, unmaintainable scripts. Length: 300–350 words.

- Required: 5 specific best practices with bad example → good example for each
- Topics to cover: shebang lines, comments, variable naming, error handling (set -e, 2>/dev/null), exit codes
- Tone: Practical and direct — written by a practitioner, not an academic

DAY 3

Log Analysis from CLI (auth.log, syslog, Apache)

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Identify the structure and key fields in auth.log, syslog, and Apache access.log

2. Chain grep, awk, sed, cut, sort, and uniq to extract attack intelligence

3. Detect brute force, privilege escalation, and web attacks from log entries

A SECURITY LOG

is a timestamped record of events on a system that allows analysts to reconstruct what happened, when, who was responsible, and what impact it had.

The Three Most Important Log Files

Log File	Location	What It Records	Key Attack Evidence
auth.log	/var/log/auth.log	SSH logins, sudo, su, PAM authentication	Failed passwords, privilege escalation, new user creation
syslog	/var/log/syslog	System-wide events, service starts/stops, kernel	Unexpected service crashes, cron jobs, kernel errors
Apache access.log	/var/log/apache2/access.log	Every HTTP request to the web server	SQL injection attempts, directory traversal, scanner fingerprints
Apache error.log	/var/log/apache2/error.log	Web server errors and warnings	File not found (scanning), PHP errors, permission denied
kern.log	/var/log/kern.log	Kernel messages, hardware events	USB device insertion, suspicious kernel module loads

auth.log — Line-by-Line Anatomy

auth.log Entry Breakdown

```
■■■
```

PRACTICE *(2 Hours)*

LAB 3.3.1 — Full Attack Chain Detection

Objective: Identify a complete attack chain across multiple log files.

Prerequisites: Terminal open; practice log files created in Day 1 lab

Step 1 — Create Realistic Apache Log

BASH

```
cat > ~/practice_apache.log << 'EOF' 192.168.1.100 - - [15/Jan/2024:08:25:00] "GET /index.php
HTTP/1.1" 200 1234 192.168.1.100 - - [15/Jan/2024:08:25:01] "GET /admin HTTP/1.1" 403 512
192.168.1.100 - - [15/Jan/2024:08:25:02] "GET /phpmyadmin HTTP/1.1" 404 256 192.168.1.100 - -
[15/Jan/2024:08:25:03] "GET /wp-admin HTTP/1.1" 404 256 192.168.1.100 - - [15/Jan/2024:08:25:10] "GET
/login.php?id=1' OR '1'='1 HTTP/1.1" 200 5000 192.168.1.100 - - [15/Jan/2024:08:25:15] "GET
/login.php?id=1 UNION SELECT user(),password() FROM users-- HTTP/1.1" 200 8000 185.220.101.5 - -
[15/Jan/2024:08:30:00] "GET /robots.txt HTTP/1.1" 200 100 185.220.101.5 - - [15/Jan/2024:08:30:01]
"GET /../../../etc/passwd HTTP/1.1" 400 0 EOF
```

Step 2 — Identify the Attacker's Reconnaissance Phase

BASH

```
# Find all 404s (probing for hidden pages) grep ' 404 ' ~/practice_apache.log # Find all requests from
the suspicious IP grep '192.168.1.100' ~/practice_apache.log | awk '{print $7}'
```

What this tells you: The attacker probed /admin, /phpmyadmin, /wp-admin before finding the vulnerable login page. This reconnaissance phase always appears in logs before the actual attack.

Step 3 — Detect the SQL Injection

BASH

```
# Detect SQL injection patterns grep -i "'\|union\|select" ~/practice_apache.log # Get the exact
malicious URLs grep -i 'union' ~/practice_apache.log | awk '{print $7}'
```

What this tells you: The UNION SELECT attempted to extract usernames and passwords from the database. The HTTP 200 response means it likely succeeded.

Step 4 — Correlate with auth.log

Now cross-reference: did the same IP that ran SQL injection also attempt SSH brute force?

BASH

```
grep '192.168.1.100' ~/practice_auth.log grep '192.168.1.100' ~/practice_apache.log | wc -l
```

This is the multi-vector attack: web exploitation + credential brute force from the same IP. This is your incident timeline.

■ ANALYSE THIS

The HTTP response code for the SQL injection attempt was 200 (success) and the response size was 8000 bytes (much larger than normal). What does the large response size indicate? How would you determine if actual data was exfiltrated?

BUILD *(2 Hours)*

BUILD BRIEF

Write a Multi-Log Correlation Script that reads both auth.log and Apache access.log and produces a unified incident timeline.

Script requirements:

- Accept log file paths as arguments: ./correlate.sh auth.log apache.log
- Extract all events from both logs sorted by timestamp
- Flag: failed logins, successful logins, sudo usage, SQL patterns, directory traversal, 404 probing
- Produce unified output: [TIMESTAMP] [SOURCE] [TYPE] [DETAILS]
- Add colour coding: red for attacks, green for successful logins, yellow for probing

Colour in bash:

- RED='\033[0;31]' YELLOW='\033[0;33]' GREEN='\033[0;32]' NC='\033[0m'
- echo -e "${RED}[ALERT]${NC} Found attack"

Deliverable:

- Working script + demonstration on your practice log files + sample output showing a complete attack chain detected

DOCUMENT

Writing Prompt:

Write a Log Review Procedure — a Standard Operating Procedure (SOP) for a Tier 1 SOC analyst performing daily log review.

- Required: Purpose, Scope, Frequency (daily/shift), Tools Required, Step-by-step procedure (numbered), Decision tree: what to escalate vs handle vs ignore, Escalation criteria with severity levels
- Format: SOP document with version number and author — professional enough to hand to a new hire on their first day

DAY
4

Python Basics for Security + AI-Assisted Development

LEARN *(1 Hour)*

LEARNING OBJECTIVES

1. Write Python scripts to automate log analysis tasks

2. Use Python's subprocess module to run shell commands programmatically

3. Apply AI-assisted development to write and debug security scripts faster

PYTHON IN SECURITY

Python is the language of security automation. It runs on every platform, has libraries for every security task (network scanning, log parsing, API calls, cryptography), and is readable enough that even non-programmers can understand what a script does.

Why Python After Bash?

Task	Bash Approach	Python Approach	When to Use Python
Count log entries	grep | wc -l	len(open('log').readlines())	When you need the number in a larger calculation
Parse structured data	awk -F: '{print $1}'	csv.reader() or json.load()	JSON, CSV, XML — bash struggles here
API calls	curl + manual parsing	requests.get() + .json()	When you need to call REST APIs (threat intel, SIEM)
Complex logic	Nested if/else — gets messy	Clean functions and classes	Any logic over 3 levels deep
Scheduling/repetition	Cron + bash	APScheduler or loop with sleep()	When you need adaptive timing
Reporting	echo to file	f-strings, jinja2, PDF libraries	Professional formatted reports

Python Security Essentials — Quick Reference

PYTHON

```
# ■■ Reading and Parsing Log Files ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ with open('/var/log/auth.log', 'r', errors='ignore') as f: lines = f.readlines() # Filter for failed logins failed = [line for line in lines if 'Failed password' in line] print(f'Failed logins: {len(failed)}') # ■■ Extracting IP Addresses ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ import re ip_pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b' ips = [re.search(ip_pattern, line).group() for line in failed if re.search(ip_pattern, line)] # ■■ Counting Occurrences ■■■■■■■■■■■■■■■■■■■
```

PRACTICE *(2 Hours)*

LAB 3.4.1 — Python Log Analyser

Objective: Rewrite the bash brute-force detector from Day 2 in Python.

Prerequisites: Python 3 installed; practice_auth.log from Day 1

Step 1 — Write the Basic Log Reader

PYTHON

```
# Create file: ~/log_analyser.py #!/usr/bin/env python3 """ TechRise 3.0 — Week 3 Log Analyser: Detect brute force from auth.log """ import re from collections import Counter from datetime import datetime LOG_FILE = '/home/kali/practice_auth.log' THRESHOLD = 3 # Alert if >3 failed attempts from same IP def read_log(filepath): """Read log file safely, handling encoding issues""" try: with open(filepath, 'r', errors='ignore') as f: return f.readlines() except FileNotFoundError: print(f'[ERROR] Log file not found: {filepath}') return [] except PermissionError: print(f'[ERROR] Permission denied: {filepath}') return [] lines = read_log(LOG_FILE) print(f'Read {len(lines)} lines from {LOG_FILE}')
```

Step 2 — Extract Failed Login IPs

PYTHON

```
def get_failed_ips(lines): """Extract IP addresses from failed login lines""" ips = [] ip_pattern = r'\b(?:\d{1,3}\.){3}\d{1,3}\b' for line in lines: if 'Failed password' in line: match = re.search(ip_pattern, line) if match: ips.append(match.group()) return ips failed_ips = get_failed_ips(lines) print(f'Found {len(failed_ips)} failed login attempts') # Count by IP ip_counts = Counter(failed_ips) print('\nTop attacking IPs:') for ip, count in ip_counts.most_common(5): print(f' {ip}: {count} attempts')
```

Step 3 — Add Alerting Logic

PYTHON

```
def generate_alerts(ip_counts, threshold): """Generate alerts for IPs exceeding threshold""" alerts = [] for ip, count in ip_counts.items(): if count > threshold: severity = 'CRITICAL' if count > 20 else 'HIGH' if count > 10 else 'MEDIUM' alerts.append({ 'ip': ip, 'count': count, 'severity': severity, 'type': 'Brute Force Attempt' }) return alerts alerts = generate_alerts(ip_counts, THRESHOLD) print(f'\n[ALERTS GENERATED: {len(alerts)}]') for alert in alerts: print(f" [{alert['severity']}] {alert['type']}: {alert['ip']} ({alert['count']} attempts)")
```

Step 4 — Run and Test

BASH

```
python3 ~/log_analyser.py
```

Expected output should show 1 MEDIUM alert for 192.168.1.100 (5 attempts) and analysis of all failed logins.

■ ANALYSE THIS

Your Python script took how many milliseconds to run? Your bash one-liner runs in similar time. For small log files, bash is faster to write. For logs over 100MB or when you need structured output (JSON, database), which would you choose and why?

BUILD *(2 Hours)*

BUILD BRIEF

Extend log_analyser.py into a full Security Intelligence Tool.

Add these capabilities:

- Multi-log support: accept log file path as command-line argument (import sys; sys.argv[1])
- Apache log parsing: detect SQL injection patterns (union, select, OR 1=1) in URL fields
- Attack correlation: flag IPs that appear in BOTH auth.log failures AND Apache attack patterns
- JSON report output: save findings to report.json with timestamp, alerts list, and summary
- Summary statistics: total events, total alerts, highest severity, most active attacker IP

Command-line usage should be:

- python3 log_analyser.py --auth auth.log --apache apache.log --output report.json

Deliverable:

- Working Python script + sample report.json + README explaining usage

AI policy:

- You may use AI to help with: argparse setup, regex patterns, JSON formatting. Must write the security logic (alert thresholds, correlation) yourself.

DOCUMENT

Writing Prompt:

Write a comparison: Bash vs Python for Security Automation. Audience: security team lead deciding which to use for a new log monitoring project. Length: 350–400 words.

- Required: 5 scenarios where bash wins, 5 where Python wins, a decision framework (flowchart or table), your personal recommendation with justification
- You may use AI to generate a first draft but must add your own examples from this week's labs

DAY
5

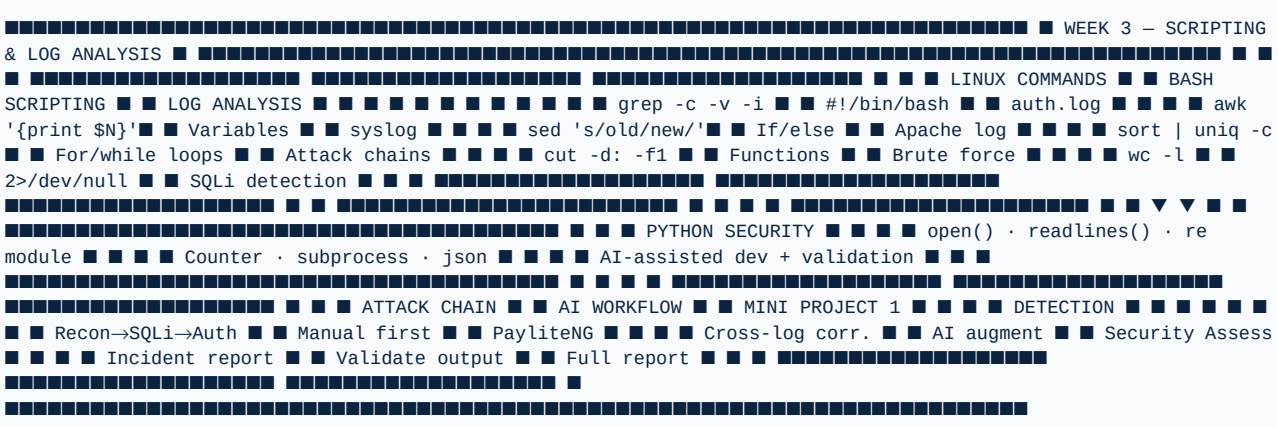
Review, Assessment & Portfolio (Mini Project 1 Submission)

LEARN *(1 Hour) — Week Review*

LEARNING OBJECTIVES

1. Demonstrate all Week 3 skills without referencing notes
2. Submit Mini Project 1 with professional documentation
3. Plan Week 4 preparation tasks

Week 3 Mind Map — Linux, Scripting & Log Analysis



PRACTICE *(2 Hours) — Skills Assessment*

LAB 3.5.1 — Command Chain Challenge

Objective: Complete log analysis tasks from memory without referencing notes.

Prerequisites: No notes — terminal only

Tasks (complete from memory):

1. From /var/log/auth.log: find all failed SSH attempts and count them in one command
2. Find the top 3 attacking IP addresses sorted by number of attempts
3. Find all successful logins and check if any IP also appears in the failed attempts
4. Find all sudo commands run in the last 24 hours
5. Write a one-liner that: reads auth.log, finds Failed password lines, extracts IPs, sorts, counts, and shows top 5

LAB 3.5.2 — Script Debug Challenge

Objective: Fix the broken security script.

Prerequisites: Terminal open; no AI for this lab

Create this broken script and fix all 5 bugs:

BASH

```
#!/bin/bash # This script has 5 bugs — find and fix them logfile = "/var/log/auth.log" # Bug 1
threshold=10 count=$(grep -count 'Failed' $logfile) # Bug 2 if [ $count -gt $threshold ] # Bug 3 echo
"[ALERT] $count failed logins" fi awk "{print $11}" $logfile | sort | uniq -c # Bug 4 echo report
saved to $REPORT_FILE # Bug 5
```

■ ANALYSE THIS

Identify all 5 bugs before running the script. For each: what type of error is it (syntax, logic, variable, command), what happens when the script runs with that bug, and what is the correct fix.

BUILD *(2 Hours) — Mini Project 1 Submission*

■ MINI PROJECT 1 — Security Assessment Report — PayliteNG (SUBMISSION)

Complete and submit your professional security assessment report for PayliteNG, the fictional Nigerian fintech company introduced in Week 2. This is your first major portfolio deliverable.

Deliverables:

- Executive Summary: 1 paragraph — overall security posture, top 3 risks, recommended priority action
- Asset Inventory: 10 assets with classification (Public/Internal/Confidential/Secret)
- Threat Assessment: 5 threats with OSI layer + OWASP + STRIDE mapping for each
- Risk Register: 5 risks with Likelihood (1–3), Impact (1–3), Score, Priority
- Vulnerability Findings: 3 specific vulnerabilities with CVE references and CVSS scores
- NIST CSF 2.0 Gap Analysis: rating for all 6 functions (1–5) with justification
- Recommendations: 5 prioritised actions with effort, impact, and 30/60/90-day timeline
- Log Analysis Appendix (NEW — Week 3 addition): add a section showing what log entries you would look for to detect attacks against PayliteNG — include 5 specific grep commands tailored to their environment
- AI Usage Appendix: list every AI tool used, the prompt given, and how you validated the output

Tools: All frameworks from Weeks 1–3 · NVD/CVE · OWASP Testing Guide · Python/Bash (for log section) · AI tools (documented)

Mini Project 1 — Submission Requirements

- Format: PDF report (minimum 15 pages excluding appendices)
- Submission: PDF + supporting scripts folder (your bash/Python scripts from Week 3)

- Evaluation criteria: Accuracy of framework application (40%), Depth of analysis (30%), Professional writing quality (20%), AI usage documentation (10%)
- Feedback: You will receive written feedback before Week 4 begins

DOCUMENT *(1 Hour) — Week Summary*

DOCUMENT

Writing Prompt:

Write your Week 3 learning reflection addressed to your Week 12 self. Length: 300–400 words.

- Three specific commands or scripts you can now write from memory (demonstrate by including the actual code)
- The hardest concept this week and how you worked through it
- One real security incident you read about this week that connects to log analysis
- How your bash script from Day 2 compares to your Python script from Day 4 — which was easier to write and why
- Prediction: Which log file will be most important in your Week 6 web application security lab?

Weekly Checklist (Student-Facing)

- ■ I can write grep | awk | sort | uniq -c to find top attacking IPs from memory
- ■ I can explain the difference between grep -c, grep -v, and grep -i
- ■ I have written a bash script with variables, if/else, and a loop that runs without errors
- ■ I can read and interpret auth.log entries — identifying timestamp, service, event, and source IP
- ■ I can detect brute force, successful login, and privilege escalation from auth.log
- ■ I can detect SQL injection, directory traversal, and scanner activity from Apache access.log
- ■ I have written a Python script that reads a log file and generates alerts
- ■ I have used AI to help with regex or syntax, validated the output, and documented the process
- ■ My Mini Project 1 is submitted as a PDF with all required sections
- ■ I have reviewed my Mini Project 1 AI appendix and confirmed every AI fact is verified

Skills Verification

Skill	How to Demonstrate It	Pass Standard
grep pipeline	Given auth.log, find top 5 attacking IPs in one command chain	Correct output within 2 minutes, no notes
awk field extraction	Extract fields 1, 3, 9, 11 from a given auth.log line	Correct awk '{print $1,$3,$9,$11}' syntax
sed transformation	Replace all occurrences of 'Failed' with '[ALERT]' in log output	sed 's/Failed/[ALERT]/g' correct syntax
Bash script	Show security_check.sh running without errors on Kali	Script completes, report file generated
Attack detection	Given an Apache log with SQLi, find the malicious requests	Correct grep pattern, malicious lines identified
Python log parsing	Show log_analyser.py producing alert output	Script runs, alerts generated for threshold violations
AI validation	Show AI prompt + response + your correction or confirmation	At least one finding documented where you improved AI output

Self-Assessment Quiz

Instructions

- Answer all questions without looking at notes.
- Write answers on a separate sheet.
- Key is in Instructor Notes.

Command Output Interpretation

1. You run this command:

BASH

```
grep 'Failed password' /var/log/auth.log | awk '{print $11}' | sort | uniq -c | sort -rn | head -5
```

The output shows:

```
47 45.33.32.156 23 192.168.1.100 8 10.0.0.1
```

- What does this output mean in plain English?
- Which IP is most suspicious and why?
- What would you do next as a security analyst?

Script Analysis

2. Identify what this script does and any security concerns:

BASH

```
#!/bin/bash for ip in $(grep 'Failed' /var/log/auth.log | awk '{print $11}' | sort -u); do
count=$(grep -c $ip /var/log/auth.log) if [ $count -gt 10 ]; then iptables -A INPUT -s $ip -j DROP fi
done
```

- What does this script do?
- What is the security risk of running this automatically?
- What legitimate IP could get blocked by this script?

Python Challenge

3. Write Python (from memory) to: read ~/practice_auth.log, find all lines with 'Accepted password', extract the IP address and username, and print each as: 'Successful login: [username] from [IP]'

Attack Chain Analysis

4. You have two log files. In auth.log: 45 failed SSH logins from 45.33.32.156, then 1 successful login from 45.33.32.156, then: sudo: user kali, COMMAND=/bin/bash. In Apache access.log: 200 status for /login.php?id=1 UNION SELECT user(),pass() FROM users. Build the complete attack timeline. What MITRE ATT&CK; techniques are represented? What should the immediate response be?

AI Critical Thinking

5. You ask an AI: 'Write a bash script to block all IPs with more than 5 failed logins using iptables.' The AI provides a script. Before running it: list 5 things you must check. What could go wrong if you run it blindly?

Additional Resources

Command References

- GNU grep manual: https://www.gnu.org/software/grep/manual/grep.html
- GNU awk (gawk) reference: https://www.gnu.org/software/gawk/manual/gawk.html
- sed one-liners reference: https://www.pement.org/sed/sed1line.txt

Log Analysis

- SANS Log Analysis Poster (Free PDF): https://www.sans.org/posters/log-management-poster/
- Linux auth.log explained — Red Hat docs: https://access.redhat.com/documentation/
- Apache Log Format reference: https://httpd.apache.org/docs/current/logs.html

Python Security

- Python for Hackers — Automate the Boring Stuff (Free): https://automatetheboringstuff.com
- Real Python — File handling tutorial: https://realpython.com/read-write-files-python/
- Python re module documentation: https://docs.python.org/3/library/re.html

Practice Labs

- TryHackMe — Linux Fundamentals Part 3 (log analysis): https://tryhackme.com/room/linuxfundamentalspart3
- TryHackMe — Investigating with ELK 101 (log analysis): https://tryhackme.com/room/investigatingwithelk101

MITRE ATT&CK; — Week 3 Relevant Techniques

- T1110 — Brute Force: https://attack.mitre.org/techniques/T1110/
- T1548.003 — Sudo Abuse: https://attack.mitre.org/techniques/T1548/003/
- T1190 — Exploit Public-Facing Application: https://attack.mitre.org/techniques/T1190/

Looking Ahead — Week 4

Preview — What's Coming in Week 4

- Reconnaissance & Scanning — the first phase of penetration testing
- Nmap: the most important scanning tool in cybersecurity — you will use it every single day
- OSINT (Open Source Intelligence) — gathering information from public sources
- Passive vs Active reconnaissance — legal boundaries and real techniques
- Your log analysis skills become essential: Nmap generates log entries that defenders analyse

What to Review Before Week 4

- ss -tuln — you will compare your scan results against what ss shows locally
- The 10 critical ports from Week 2 — Nmap will find them and you need to know what they mean
- Network IP ranges from Week 1 — you need to understand CIDR notation for scan targets

What to Install Before Week 4

- Verify Nmap is installed: nmap --version (should already be on Kali)
- Verify Metasploitable2 VM is downloaded and importable into VirtualBox
- Create a labs/ directory: mkdir ~/week4_labs
- Read: https://nmap.org/book/man.html (the Nmap man page — 15 minutes)

■ Week 4 Legal Reminder

Week 4 involves active scanning. You will ONLY scan your own lab VMs. Scanning any system you do not own or have explicit written permission to scan is illegal under Nigeria's Cybercrimes Act 2015. The lab environment (Kali + Metasploitable2 on your own machine) is your authorised target.